

SepsisCare 期末汇报中文演讲稿

ICU 脓毒症风险分层、动态表型与多端软件交付

汇报日期：2026 年 6 月 10 日 适配正式 17 页 PPT

项目资料来源：SepsisCare final delivery package, report dated 2026-06-03, delivery refreshed 2026-06-07

建议节奏：如果老师给 10 到 12 分钟，每页平均 35 到 45 秒，研究部分稍慢，安全与部署部分稍快。如果只剩 8 分钟，优先保留第 1、2、3、4、7、10、11、14、16、17 页，把第 5、6、8、9、12、13、15 页压成过渡句。

目录

1	开场	2
	第 1 页：SepsisCare 总览	2
	第 2 页：问题与边界	2
2	数据与研究路线	2
	第 3 页：数据基础	2
	第 4 页：S0 到 S7 研究路线	2
	第 5 页：S0 到 S1.5：表示基础	3
	第 6 页：S2 到 S5：从表型到实时使用	3
	第 7 页：S6 到 S7：最终鲁棒训练	3
	第 8 页：动态表型	4
	第 9 页：最终 S7 模型	4
	第 10 页：关键结果与证据边界	4
3	软件系统与交付	4
	第 11 页：多角色产品体验	4
	第 12 页：软件架构与 API 流	5
	第 13 页：网络安全加固	5
	第 14 页：数据污染与 Artifact 完整性	5
	第 15 页：部署编排	5
	第 16 页：最终交付物	6
4	收束与答辩准备	6
	第 17 页：限制与下一步	6
5	可删减版提示	6

1 开场

第 1 页：SepsisCare 总览

本页主旨：先把项目定位讲清楚：这是一个从 ICU 时间序列研究走到可安装软件和可验证交付包的课程项目。

提示：语速放稳，第一句话不要急着解释模型名词。

讲稿：各位老师好，我今天汇报的项目是 SepsisCare。它的主题是 ICU 脓毒症风险分层、动态表型学习，以及可部署的多端软件系统。

我想先强调项目边界：SepsisCare 不是临床诊断设备，也不会替代医生决策。它是一个课程项目，目标是把 ICU 时间序列研究原型，扩展成一个可以安装、可以演示、可以通过脚本验证的工程系统。

整个项目最终交付了四类内容：第一是研究模型，包括 S7 动态表型模型；第二是可运行的多端应用；第三是模型服务、部署脚本和训练终端；第四是文档、安全审计和 residual-risk 边界说明。几个核心数字可以作为总览：S7 声明训练患者数是 347,634，表型 Macro F1 是 0.956，死亡风险 AUROC 是 0.848。这些指标支持课程演示和工程验证，但我不会把它们表述成临床外部验证结论。

第 2 页：问题与边界

本页主旨：说明“为什么做”和“不能声称什么”，让后面的技术路线显得可信。

讲稿：SepsisCare 选择脓毒症，是因为它在 ICU 中变化很快，而且病情不是一个静态分数就能完整描述的。一个患者在 48 小时内，可能从相对稳定转向循环休克，也可能因为机械通气、乳酸、血压和感染指标的变化进入另一个风险状态。所以我希望系统不仅输出一个风险值，还能表达轨迹变化、表型转移和相似病例。

这个项目分成三层。第一层是研究层：从脱敏 ICU 时间序列中学习患者表示和窗口表示。第二层是工程层：把模型包装成医生端、研究端、家属端、管理员端和训练终端都能访问的软件系统。第三层是保证层：加入 token 鉴权、SSRF 防护、数据审计、artifact manifest 和部署验证脚本。

这里最重要的边界是：系统输出只能用于课程演示和工程验证，不能作为自主临床医嘱、最终诊断，也不是受监管医疗器械声明。这个边界贯穿了我的报告、PPT、应用界面和最终交付文档。

2 数据与研究路线

第 3 页：数据基础

本页主旨：四个 ICU 来源被统一到 48 小时接口，最终只打包脱敏演示数据，不打包受限原始库。

讲稿：数据层面，我把四类 ICU 来源统一到了一个 48 小时时间序列接口。PhysioNet 2012 是主表型来源，包含 11,986 名患者；PhysioNet 2019 提供 sepsis 辅助来源，包含 40,331 名患者；MIMIC-IV 作为外部来源，包含 94,458 名患者；eICU 是更大的多中心来源，包含 200,859 名患者。最终 S7 声明训练患者总数是 347,634。

统一接口中，模型看到的不只是数值，还包括观测掩码、静态特征、干预代理变量、可用标签和数据来源 provenance。不同数据库标签并不完全一致，所以训练时用 masked loss，只在某个任务标签可用时才计算对应损失，而不是强行制造假标签。

最终交付包不包含受限原始 ICU 数据库。它只包含模型报告、权重、读出器、部署代码和脱敏演示 runtime 数据。这一点对项目的伦理边界和交付安全都很重要。

第 4 页：S0 到 S7 研究路线

本页主旨：把研究路线讲成一个逐步解决问题的过程，而不是堆模型名字。

讲稿：研究路线从 S0 到 S7，每一步都是为了解决前一步暴露出来的具体问题。

S0 做数据标准化，把不同 ICU 数据对齐到 48 小时张量，并做患者级切分，避免同一患者跨训练和验证泄漏。S1 做 masked reconstruction，让模型先学会稀疏 ICU 时间序列的结构。S1.5 加入对比学习，让同一患者的增强视图在表示空间中更稳定。

中间的 S2 到 S5 把表示变成可解释的动态状态。S2 用滚动窗口发现 P0 到 P3 的表型，S3 检查 Center A 到 Center B 的跨中心稳定性，S3.5 做死亡风险读出和校准，S5 让模型适应实时场景中“只看到当前小时以前数据”的情况。

最后 S6 和 S7 是最终模型选择。S6 引入缺失感知 GRU，把缺失本身当作临床信号；S7 加入源均衡采样、GroupDRO、表型交叉熵、监督对比学习和多个辅助任务，形成最终可部署模型。我的选择标准不是单一指标最高，而是这个阶段是否改善了解释、迁移、实时使用或交付证据。

第 5 页：S0 到 S1.5：表示基础

本页主旨：解释为什么没有一开始就做死亡分类，而是先做稳定表示。

讲稿：这一页对应研究的第一块：先把 raw ICU records 变成可以信任的患者轨迹表示。

S0 的核心是输入纪律。每个 ICU stay 被对齐到入 ICU 后的 1 到 48 小时，形成连续值 X 、观测掩码 M 、静态特征 A 、干预代理变量 P 和结局 Y 。标准化统计量只从训练患者估计，这样可以减少信息泄漏。

S1 用 masked reconstruction：随机隐藏已经观测到的值，让编码器去重构它们。这个阶段的意义是，在依赖结局标签之前，先验证 ICU 时间序列本身是否能被模型捕捉。S1.5 再做 contrastive augmentation，把同一患者的两种增强视图拉近，让表示空间更稳定。

所以我没有直接跳到死亡分类器，而是先回答一个更基础的问题：这个时间序列表示是否足够稳定，能不能支撑后面的表型发现和解释面板。

第 6 页：S2 到 S5：从表型到实时使用

本页主旨：动态表型是研究指标和软件解释之间的桥。

讲稿：第二块研究把 embedding 转换成动态表型，并进一步考虑实时使用。

S2 把每个 48 小时轨迹切成 5 个 24 小时滚动窗口，然后对每个窗口编码并聚类成 P0 到 P3 四类表型。这样，一个患者不再只有一个静态标签，而是有一个随时间变化的表型序列。S3 则做跨中心检查：在 Center A 训练的表型结构，迁移到 Center B 后，还要看 prevalence、死亡率排序、最高风险状态和转移模式是否稳定。

S3.5 把 embedding 和统计特征结合起来做风险读出，并比较分类器和校准策略。S5 进一步考虑真实演示中不会一次性看到完整 48 小时数据，所以加入 horizon augmentation，让模型在第 h 小时只使用 h 小时以内的信息，同时用阈值、最小历史长度、连续触发和冷却时间控制报警。

这一阶段的结果是：表型状态成为研究模型和软件解释之间的桥梁。软件里能解释的不只是“高风险”，还包括“患者正在从哪个表型向哪个表型移动”。

第 7 页：S6 到 S7：最终鲁棒训练

本页主旨：最终模型把缺失机制、数据源差异、表型监督和辅助临床任务放到同一个训练目标中。

讲稿：第三块研究是最终模型。ICU 数据的缺失不是普通表格缺失，它往往和临床采样频率、病情严重程度和医院流程有关。因此 S6 的 missingness-aware GRU 不只输入数值，还输入观测掩码、干预代理变量和距离上次观测的时间差，让模型显式感知缺失模式。

在跨源训练上，我尝试了 source balance、DANN、CORAL、GroupDRO、GRU-D 风格输入和 ensemble。结果显示，简单追求域不变并不稳定，因为某些域差异本身包含有用的临床信息。最终 S7 选择源均衡采样和 GroupDRO，同时加入主源表型 CE、监督对比学习，以及 LOS、机械通气、RRT 等辅助任务。

最终输出包括轨迹编码器、表型读出器、转移矩阵和模型服务包。核心结果是：347,634 声明训练患者，31,319 个模型参数，encoder Macro F1 0.971，平滑后的表型 Macro F1 0.956，下一时段机械

通气 AUROC 0.968。这里我再次强调：这是课程交付模型和 smoke-test target，不是临床 held-out 外部验证声明。

第 8 页：动态表型

本页主旨：用表型轨迹把模型从黑盒分数变成可沟通的临床状态序列。

讲稿：这一页更直观地解释动态表型。每个患者 48 小时被切成 5 个滚动窗口，每个窗口进入编码器，再被分配到 P0 到 P3 的某个状态。这样我们可以看到一个患者在时间上的状态转移，而不是只在最后给一个分数。

在表型解释中，P2 被保留为最高风险且相对稳定的状态。这个结果对软件很关键，因为医生端和研究端需要的不只是模型输出，还要知道这个风险是否来自循环休克、炎症高反应、呼吸衰竭，或者相对低危稳定状态。

因此，动态表型承担了一个 communication layer 的角色：它把时间序列 embedding 变成可以在床旁解释、可以检索相似病例、也可以和家属沟通时简化说明的状态语言。

第 9 页：最终 S7 模型

本页主旨：讲清楚最终模型为什么适合部署：小模型、多任务、可解释状态读出。

讲稿：最终 S7 模型的主体是 missingness-aware GRU，加上 source-balanced GroupDRO、phenotype CE 和 supervised contrastive regularization。相比更复杂的大模型，这个模型参数量只有 31,319，在课程交付中更容易部署、解释和打包。

它的设计选择是把多个目标合在一起：死亡风险、表型一致性、下一时段机械通气、剩余 ICU 住院时长，以及动态转移。这样部署时，系统不只是给一个分类结果，而是可以服务多个界面：医生端看风险和轨迹，研究端看表型和相似病例，训练终端看模型状态和 artifact。

所以这一页我想表达的是：最终模型不是为了追求单点分数，而是为了支撑一个完整软件系统中的多个 workflow。

第 10 页：关键结果与证据边界

本页主旨：主动说明指标意义和限制，比被老师追问时再解释更稳。

讲稿：这一页是关键结果。最终 S7 在课程演示监控切分上，encoder 加 transition 的 Macro F1 是 0.956，full-patient match rate 是 0.832，死亡 AUROC 是 0.848，下一时段机械通气 AUROC 是 0.968，剩余 LOS MAE 是 2.955 小时。

这些结果说明，模型能够较好复现动态表型轨迹，并保留死亡、通气和住院时长等辅助任务能力。和早期方法相比，规则 baseline 只有 0.142，S6 v2 约 0.821，GroupDRO 约 0.806，而 S7 encoder 达到 0.971，S7 smooth 是 0.956。

但是证据边界必须说清楚：S7 监控切分用于课程交付、模型比较和部署 smoke，并且与训练患者存在重叠，因此不能写成独立 held-out 外部临床验证。对本项目来说，最强证据是表型一致性加上 API/model smoke tests，而不是临床投产证明。

3 软件系统与交付

第 11 页：多角色产品体验

本页主旨：从“模型能跑”转向“用户能用”，展示医生、研究、家属、管理员四类工作流。

讲稿：从产品角度，SepsisCare 把模型包装成多角色 workflow。医生端主要看当前患者队列、风险解释、48 小时趋势、动态表型和模型状态。研究端提供历史 ICU 病例库、细节面板、相似病例和模型实验输出。家属端接入 AI 解释，但只用于非诊断性沟通，避免替代医生说明。管理员端和训练终端则负责服务状态、模型配置、日志、下载权重和继续训练。

这个设计的核心是：同一个模型服务输出，不同角色看到的是不同粒度的信息。医生需要快速判断风险，研究者需要追溯证据，家属需要自然语言解释，管理员需要知道服务有没有在线、模型有没有更新。

所以我不是只交付一个 notebook 或一个模型权重，而是把它做成了可安装、可启动、可演示的系统。

第 12 页：软件架构与 API 流

本页主旨：稳定 API 合同比模型文件本身更适合支撑多端交付。

讲稿：系统架构采用多端客户端加标准 HTTP API 加可替换模型服务的结构。macOS、Windows、Android 和 Web 共享一套主要 API 逻辑，客户端不直接读取模型文件，也不直接访问 DeepSeek。所有请求先进入 `server.py`，再由后端分发到患者、历史库、诊断、AI、训练终端或模型服务。

这样做有两个好处。第一，客户端可以更轻，不需要每个平台都实现一遍模型逻辑。第二，模型服务可以替换：课堂演示时可以用本地 packaged backend，部署演示时可以把训练终端动作转发到远端 model service。

这也是我在工程层最重视的一点：可部署系统需要稳定 contract，而不只是一个模型权重文件。

第 13 页：网络安全加固

本页主旨：把演示便利性和远端敏感 API 边界分开。

讲稿：安全部分的第一块是网络边界。早期演示系统更偏向本地可访问，但最终交付时，我把远端敏感 API 变成 fail-closed。也就是说，训练命令、artifact、审计、历史导出等敏感路径，在远端访问时必须要有强 Bearer token；如果没有配置 token、token 太弱、或者请求没有授权头，就返回 403 或 401。

前端也做了调整。API token 不再长期保存在 `localStorage`，而是保持在 session-scoped 状态；启动 URL 里的 token 参数会被清理。DeepSeek base URL 和训练终端 cloud URL 也经过 SSRF-resistant 约束，拒绝 metadata、link-local、credential-bearing 等危险地址。

另外，123123 只被定义为客户端演示门，不是生产 API 安全机制。即使输入这个演示密码，也不能绕过远端敏感 API 的 Bearer token 边界。正式远端部署仍然应该放在 VPN 或 TLS 后面，并配合 token rotation。

第 14 页：数据污染与 Artifact 完整性

本页主旨：交付包不仅要能跑，还要能证明没有明显数据污染、密钥泄漏和 artifact 损坏。

讲稿：安全保证的第二块是数据污染和 artifact 完整性。我把检查分成四层。

第一层是 runtime data audit，检查 provenance、直接标识符、重复 ID、孤立 detail、临床数值范围等。第二层是 sensitive surface audit，对模型包、后端、Web、Windows、iOS、Android 六个发布面扫描高置信密钥和 PHI-like 文本。第三层是 training metadata audit，检查四个来源、声明训练人数、target run 和 split policy，避免把 all-source monitoring split 误宣传成外部验证。第四层是 artifact verifier，下载包包含 `MANIFEST.sha256`，可选 HMAC，用于验证每个文件和整个 ZIP。

当前审计结果中，runtime fatal violations 是 0，sensitive-data findings 是 0，training metadata violations 是 0，声明训练患者数是 347,634。这个结论仍然是课程 release hygiene，不等于正式 HIPAA expert determination，也不替代 raw-source data governance。

第 15 页：部署编排

本页主旨：把部署从手写命令变成可测试、可拉回 artifact、可说明边界的流程。

讲稿：部署部分从最初的本机 demo，扩展成 laptop-to-remote-server 的编排流程。完整流程包括：先跑 pre-release gate，再做 SSH preflight，然后 rsync 同步模型服务包，远端执行 one-click doctor/start/smoke，Mac 侧验证 API forwarding 到 remote server，最后拉回并验证最新 artifact。

为了减少运维风险，break-glass 跳过预发布检查必须写理由，并记录 owner-only 审计事件；

dry-run 日志会 redacted Bearer token; artifact verifier 支持 HMAC。这样部署不是一串临时命令，而是可以被测试和复核的流程。

这里的边界也需要讲清楚：本地 app、API、model service 和部署脚本已经可复现并通过验证；但当前独立 remote server 的完整端到端实机证据仍然需要补齐。因此我会说“部署编排已就绪并验证了本机/ROG 路径”，不会说所有生产远端环境都已经完成临床级上线。

第 16 页：最终交付物

本页主旨：交付被拆成软件、模型、文档和保证脚本，方便验收和复现。

讲稿：最终交付包分为四部分。第一部分是软件安装和源码，包括 macOS app、Windows installer、Android/Web 资源、启动脚本和 UI smoke 覆盖。第二部分是模型部署包，包括 S7 权重、表型读出器、转移矩阵、模型服务、runtime database 和 one-click scripts。第三部分是文档，包括安装指南、论文报告、部署说明、transfer README、SHA256 manifest 和边界说明。第四部分是保证脚本，包括 pre-release gate、数据审计、敏感发布面扫描、训练元数据审计和 artifact verifier。

这样拆包的好处是，安装包不需要携带原始数据和 API key，模型服务可以迁移到远端，评审时也能通过 manifest、测试日志和文档追踪每个文件的来源和验证方式。

所以 SepsisCare 的交付重点不是“我电脑上能打开”，而是“另一个人拿到交付包后，能理解、安装、验证，并知道哪些边界还没有越过”。

4 收束与答辩准备

第 17 页：限制与下一步

本页主旨：最后用诚实边界收束，体现工程成熟度。

讲稿：最后我想用三个 gap 总结限制和下一步。

第一是 validation gap。当前 S7 monitoring split 适合课程交付、模型比较和 smoke test，但它不是独立 held-out clinical validation。下一步需要构建真正外部验证切分，或者在独立医院数据上做严格验证。

第二是 deployment gap。部署脚本、本机服务和 ROG 路径已经形成可复现证据，但真正 remote server 仍需要完整的跨机器端到端 proof，包括 smoke logs、artifact verifier output 和网络连通性证据。

第三是 governance gap。虽然项目已经加入 token gate、SSRF 防护、审计、manifest 和数据最小化，但受监管使用仍需要更深的 threat model、runtime display-summary warnings 清理、密钥轮换、IRB/伦理和临床安全验证。

我的最终 takeaway 是：SepsisCare 展示了一条 course-grade 的完整路径，从 ICU 时间序列建模，到动态表型解释，再到多平台软件交付和安全证据。它是 demo-ready、evidence-backed 的系统，同时也诚实地标出了还没有被临床证明的部分。谢谢各位老师。

5 可删减版提示

8 分钟压缩策略

如果现场时间紧，可以这样压缩：

1. 第 1 页只讲项目定位和边界，少讲数字。
2. 第 3 页只保留四个数据源和 347,634 总数。
3. 第 4 到第 7 页合并成“先表示、再表型、再鲁棒训练”的一段。
4. 第 8 和第 9 页只讲一句：动态表型让模型能被软件解释。
5. 第 11 到第 12 页只讲多角色 API 架构。

- 6. 第 13 到第 15 页合并成安全、审计、部署三类证据。
- 7. 第 16 页用 20 秒说明交付包结构。
- 8. 第 17 页必须保留，因为它主动说明限制。

可能被问到的问题

问题	建议回答
为什么不用单一死亡预测模型？	因为项目目标不是只输出一个死亡概率，而是服务风险看板、动态轨迹、相似病例和家属解释。动态表型让模型输出更适合软件场景。
S7 指标能不能代表临床性能？	不能直接代表。S7 monitoring split 适合课程交付和 smoke test，和训练患者有重叠，所以我只把它作为工程验证和模型比较证据。临床性能需要独立 held-out 或外部医院验证。
安全审计做到什么程度？	实现了工程层面的 token gate、SSRF 防护、session token、HTML escaping、runtime audit、sensitive surface audit、training metadata audit 和 artifact manifest。它是 release hygiene，不是 HIPAA、IRB、FDA 或医疗器械认证。
软件系统最大的工程难点是什么？	难点不是某一个页面，而是把模型、API、客户端、训练终端、部署脚本和安全审计接成可复现链路，并且让每个交付物都有边界和验证证据。